SS 2 PYTHON PROGRAMMING

TOPIC: DICTIONARIES IN PYTHON

WEEK 1: INTRODUCTION TO DICTIONARIES

Definitions

- **Dictionary**: A Python data structure that stores data in key–value pairs.
- **Key**: A unique name used to identify a value.
- **Value**: The data stored for a key.
- **Key–Value Pair**: A combination of a key and its value.

Example Code

```
student = {
  "name": "Ada",
  "age": 15,
  "class": "SS2"
}
print(student)
```

Project (Student Bio Record)

```
student = {
  "name": "John",
  "age": 16,
  "gender": "Male",
  "class": "SS2"
}
print("Student Record:", student)
```

WEEK 2: ACCESSING DICTIONARY VALUES

Definitions

- **Accessing**: Getting a value using its key.
- **get()**: A safe method for accessing values.

Example Code

```
student = {"name": "Ada", "age": 15}

print(student["name"])

print(student.get("age"))
```

Project (Profile Viewer)

```
student = {"name": "Musa", "age": 16, "class": "SS2"}

print("Name:", student.get("name"))

print("Class:", student.get("class"))
```

WEEK 3: ADDING & UPDATING ITEMS

Definitions

- **Add**: Insert a new key–value pair.
- **Update**: Change the value of an existing key.

Example Code

```
student = {"name": "Ada"}

student["age"] = 15

student["age"] = 16

print(student)
```

Project (Student Admission System)

```
students = {}

students["ST001"] = "John"
```



```
students["ST002"] = "Aisha"
print(students)
```

WEEK 4: REMOVING ITEMS

Definitions

- **pop()**: Removes a key and returns its value.
- **del**: Deletes a key or dictionary.

Example Code

```
student = {"name": "Ada", "age": 15}
student.pop("age")
print(student)
```

Project (Withdrawal Record)

```
students = {"John": "SS2", "Aisha": "SS2"}
del students["John"]
print(students)
```

WEEK 5: LOOPING THROUGH DICTIONARIES

Definitions

- **Loop**: Repeating a block of code.
- **items()**: Returns key–value pairs.

Example Code

```
student = {"name": "Ada", "age": 15}
for key, value in student.items():
  print(key, value)
```

Project (Student Display System)

```
students = {"John": 70, "Aisha": 85}
for name, score in students.items():
```



```
print(name, "scored", score)
```

WEEK 6: DICTIONARY METHODS

Definitions

- **update()**: Adds or updates items.
- **clear()**: Empties a dictionary.

Example Code

```
student = {"name": "Ada"}
student.update({"age": 15})
print(student)
```

Project (Inventory System)

```
inventory = {"Pen": 10}
inventory.update({"Book": 20})
print(inventory)
```

WEEK 7: NESTED DICTIONARIES

Definitions

- **Nested Dictionary**: Dictionary inside another dictionary.

Example Code

```
students = {
  "ST01": {"name": "Ada", "age": 15},
  "ST02": {"name": "John", "age": 16}
}
print(students["ST01"]["name"])
```

Project (Academic Record System)

```
records = {
  "Ada": {"Math": 80, "English": 75},
```



```
    "John": {"Math": 70, "English": 68}
}
print(records)
```

WEEK 8: DICTIONARIES WITH LISTS

Definitions

- **List as Value**: Using a list inside a dictionary.

Example Code

```
student = {"name": "Ada", "subjects": ["Math", "English"]}
print(student)
```

Project (Subject Registration)

```
students = {
    "Ada": ["Math", "Biology"],
    "John": ["English", "Physics"]
}
print(students)
```

WEEK 9: DICTIONARIES WITH USER INPUT

Definitions

- **Dynamic Data**: Data entered during runtime.

Example Code

```
student = {}
student["name"] = input("Enter name: ")
student["age"] = input("Enter age: ")
print(student)
```

Project (Interactive Registration)

```
students = {}
```



```
name = input("Student name: ")
score = int(input("Score: "))
students[name] = score
print(students)
```

WEEK 10: SEARCHING & UPDATING RECORDS

Definitions

- **Search**: Checking existence of a key.

Example Code

```
students = {"Ada": 80}
if "Ada" in students:
    students["Ada"] = 85
print(students)
```

Project (Record Search System)

```
records = {"John": 70, "Aisha": 88}
name = input("Search name: ")
if name in records:
    print(name, records[name])
else:
    print("Not found")
```

WEEK 11: DICTIONARIES & FILE HANDLING (JSON)

Definitions

- **JSON**: File format for storing dictionaries.

Example Code

```
import json
student = {"name": "Ada", "age": 15}
```



```
file = open("student.json", "w")
json.dump(student, file)
file.close()
```

Project (Student Backup System)

```
import json
records = {"John": 70, "Aisha": 88}
with open("records.json", "w") as f:
    json.dump(records, f)
```

WEEK 12: MINI PROJECT (INTEGRATION)

Project: Student Management System

```
students = {}
students["Ada"] = {"age": 15, "class": "SS2"}
students["John"] = {"age": 16, "class": "SS2"}
print(students)
```

WEEK 13: FINAL PROJECT & PRESENTATION

Final Project: School Record System

```
school = {
    "Students": {"Ada": 80, "John": 70},
    "Teachers": {"Mr A": "Math"}
}
print(school)
```

CODEANT